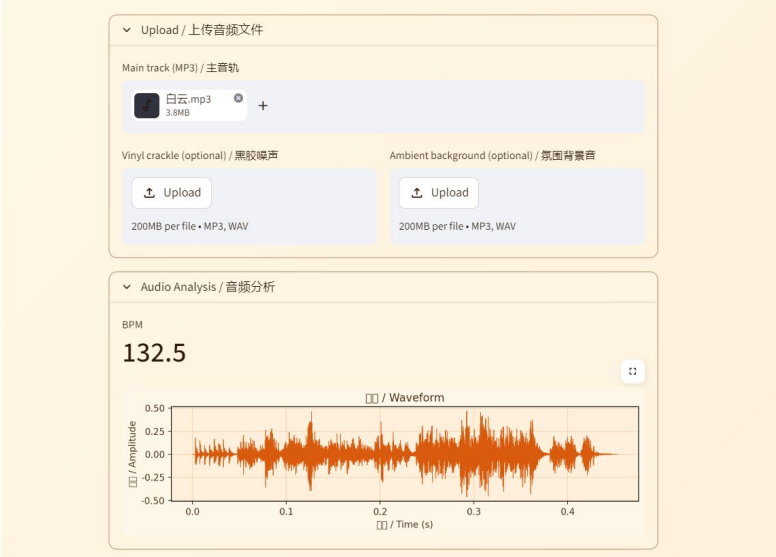
Remix Lab Development Report

Remix Lab is a web-based audio remixing tool. Without installing any professional software, you can just open the website, upload a song, and freely create remixed versions with pitch shifting, tempo changes, reverb, and various effects. The multiple presets I built can turn an ordinary song into Lo-fi retro style, a nightclub accelerated version, a muffled underwater feel, the sound of an old radio, and more.

Main webpage

The reason I wanted to build this is that I found online remixing tools were either too limited in features or required registration and payment, while professional software was too cumbersome to download and use—too high a barrier for casual enthusiasts. So I wanted to write a page myself, bring professional audio processing libraries into a webpage, and provide basic functionality with simple, clear buttons to control it. That's how Remix Lab came to be.

The entire backend audio processing logic of this project is written in Python. The web interface uses Streamlit—with this Python framework, I can skip HTML, CSS, and JavaScript and still build a complete frontend interface. For audio processing, I used three important libraries. Pydub handles MP3 reading, speed adjustment, audio cutting, and background track overlay. Spotify Pedalboard is Spotify's official open-source effects library, handling pitch shifting, reverb, distortion, delay, chorus, phaser, compressor, low-pass and high-pass filtering. Librosa analyzes audio and automatically detects BPM tempo. The waveform is drawn with Matplotlib, letting users see the shape of the sound.



Audio Analysis

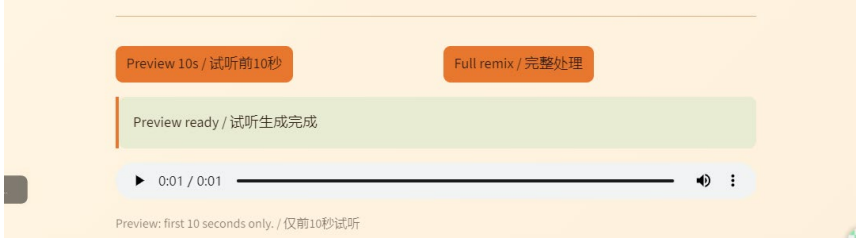
Although I used AI tools for assistance with coding, the functional design of the website was completely my own work. I designed all 11 preset effects and built a complete audio processing pipeline—from upload to tempo change, effects, EQ, background overlay, volume normalization, and export. I also implemented a background audio overlay system that supports vinyl crackle and ambient sounds, with automatic looping, automatic length alignment, and independent volume control.

Partial Code

After putting it into actual use, I ran into a few tricky problems. One was the unbearably long processing time for lengthy audio—a five-minute song took forever to fully process, which was especially painful during debugging. So in version 1.2, I added a preview mode that processes only the first 10 seconds, cutting the wait time from minutes down to a few seconds. Another issue was background tracks not aligning in length, so I wrote an automatic looping logic that repeats the background audio until it matches the main track's length, then trims it.

I also added a progress callback system that displays loading, processing, normalizing,

along with a percentage progress bar. I implemented custom preset saving and loading, so users can tweak parameters and save them for later. The color scheme was also my own design. You could say that aside from the underlying audio libraries and the Streamlit framework itself, almost every function and user experience detail was my independent creation.



Preview Function

Below I'll briefly demonstrate how to use the website. The deployed URL for Remix Lab is simple and professional. Choose an MP3 file to upload, and optionally add an ambient background track. Then select a remix mode from the dropdown—there are four main categories: Lo-fi, Chipmunk, Nightcore, and Themed Presets. If you choose Themed Presets, eight themes appear for you to pick from, and you can also check "Random Theme" for a different surprise each time. In non-themed modes, you can manually fine-tune pitch shift, playback speed, reverb strength, bass boost, and treble cut. Once adjusted, you can first try the 10-second preview, hear the result within seconds, and if you like it, proceed to full processing. Finally, download the result as either MP3 or WAV.

Mix setting

Building this tool gave me a deeper understanding of real-time audio processing and web application deployment. The computational load is significant when combining multiple audio effects, and the preview mode saved me a massive amount of debugging time. Troubleshooting compatibility issues also taught me how to handle differences across environments. I hope Remix Lab can become a truly practical tool, helping content creators, music lovers, and casual users who don't want to install complex software to quickly produce great-sounding remixes.

In the future, I plan to support more audio formats such as WAV, FLAC, and OGG; add visual effect-chain editing that lets users drag and drop effects to reorder them; implement simple four-track mixing; and explore integrating AI source separation to split vocals from instrumentals.